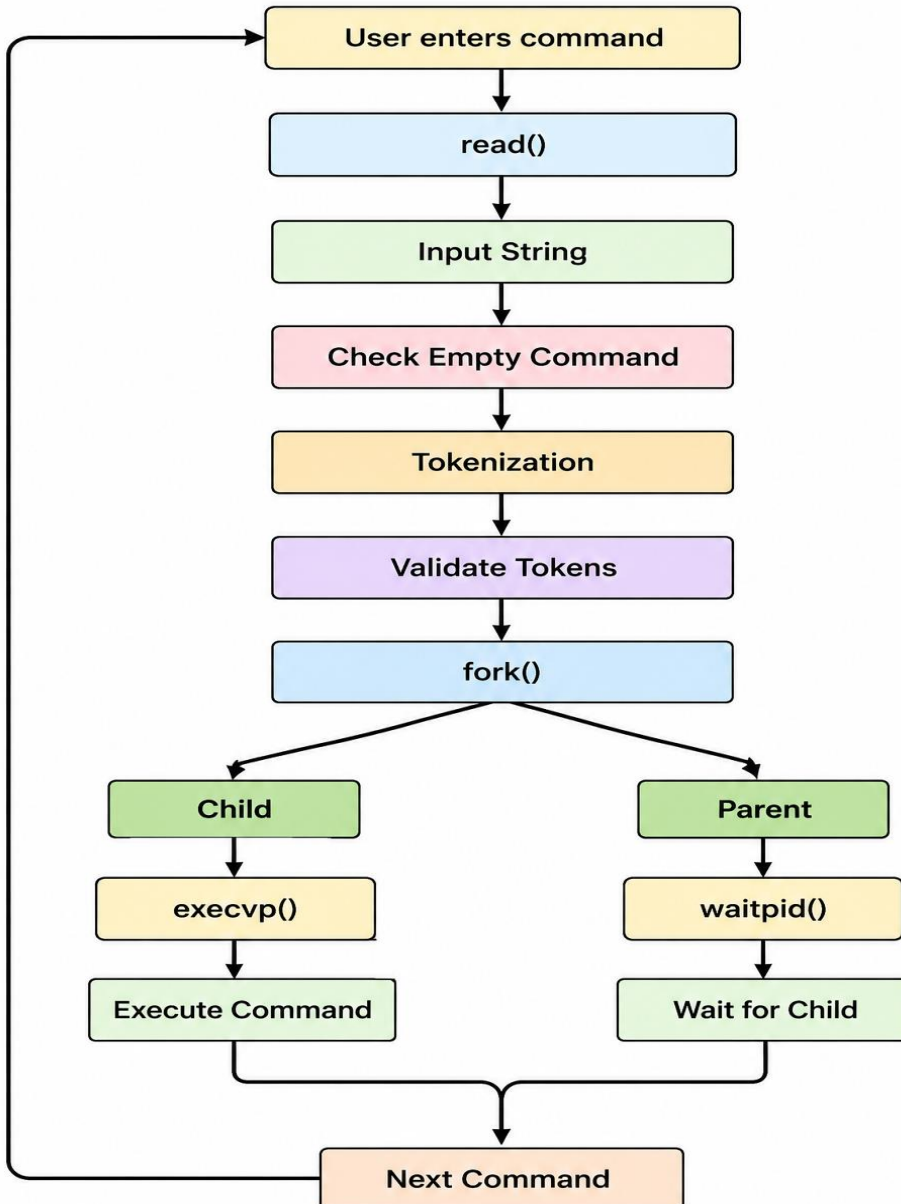


WEEK 4

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

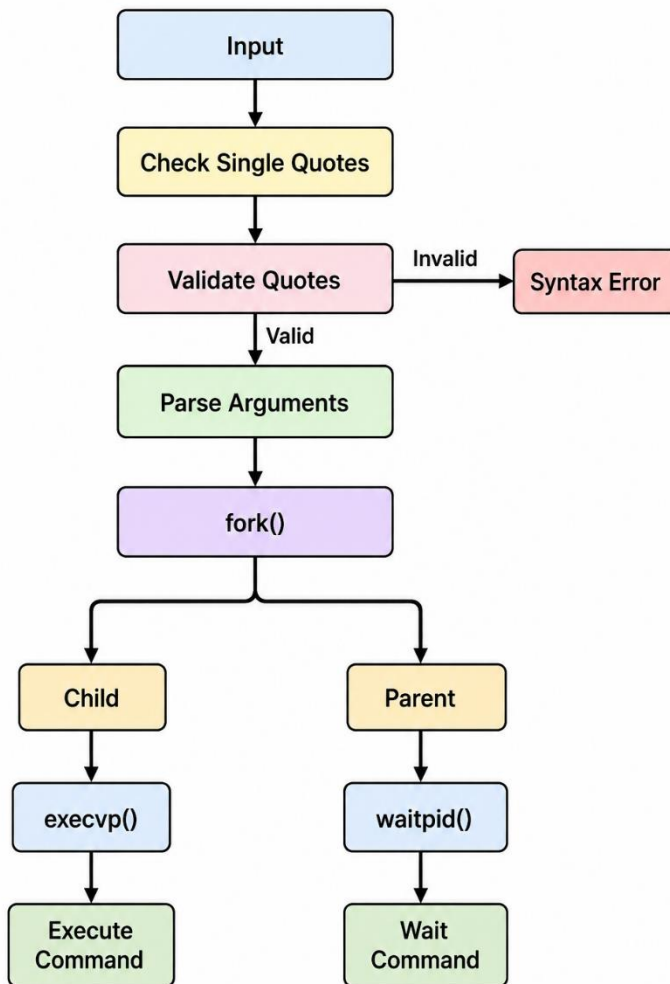
Flow Chart:



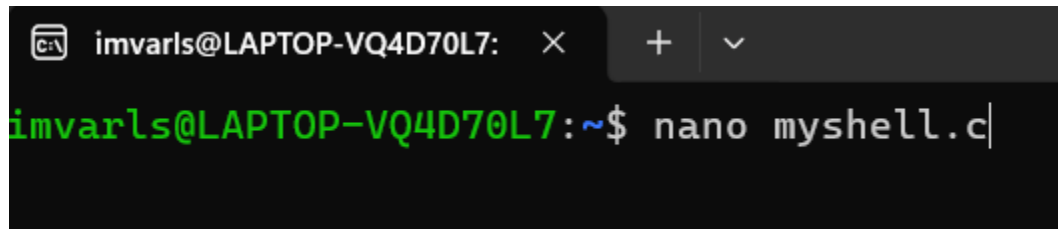
Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

Flow Chart:

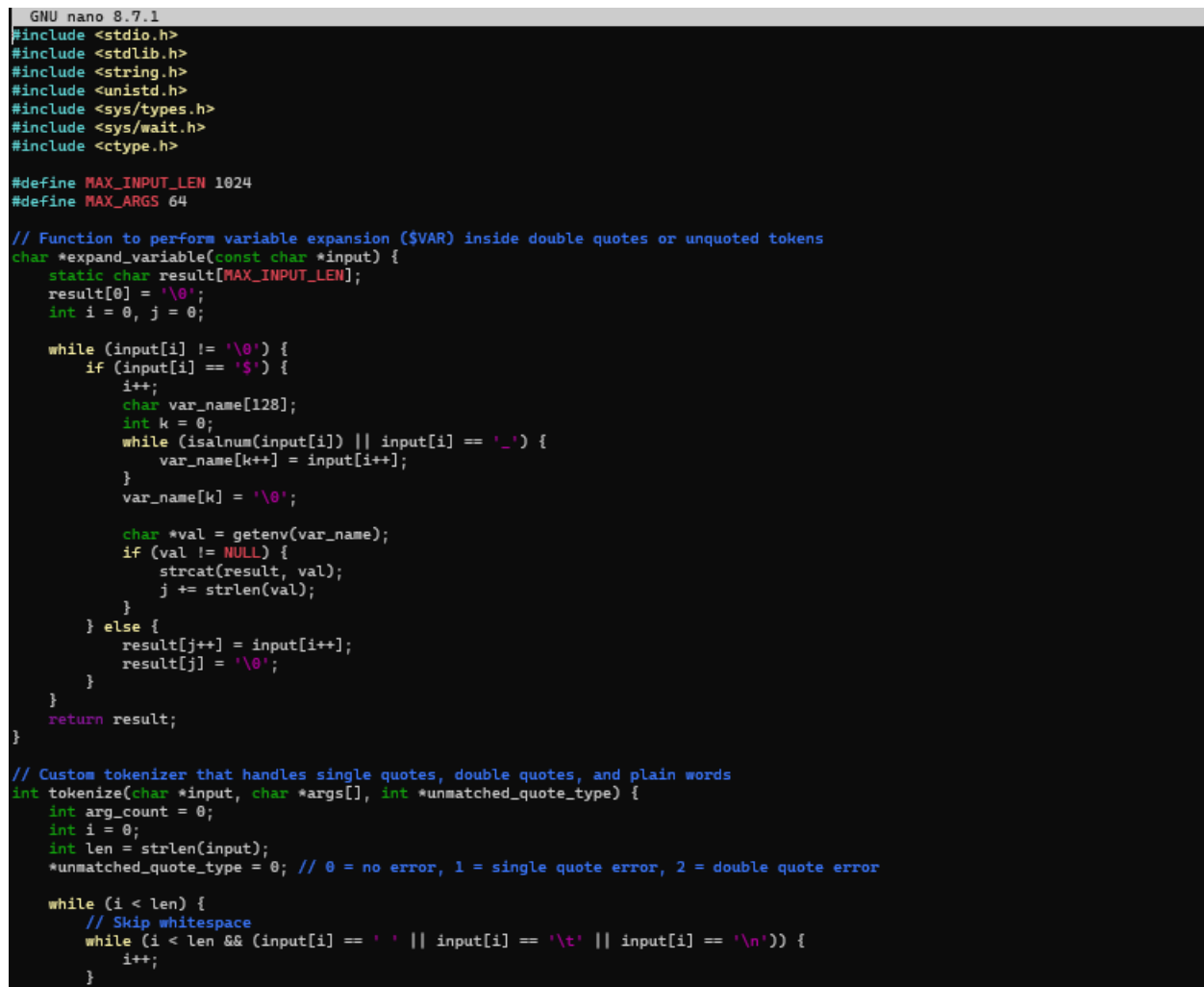


CREATE FILE



```
imvarls@LAPTOP-VQ4D70L7: ~$ nano myshell.c
```

CODE



```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <ctype.h>

#define MAX_INPUT_LEN 1024
#define MAX_ARGS 64

// Function to perform variable expansion ($VAR) inside double quotes or unquoted tokens
char *expand_variable(const char *input) {
    static char result[MAX_INPUT_LEN];
    result[0] = '\0';
    int i = 0, j = 0;

    while (input[i] != '\0') {
        if (input[i] == '$') {
            i++;
            char var_name[128];
            int k = 0;
            while (isalnum(input[i]) || input[i] == '_') {
                var_name[k++] = input[i++];
            }
            var_name[k] = '\0';

            char *val = getenv(var_name);
            if (val != NULL) {
                strcat(result, val);
                j += strlen(val);
            }
        } else {
            result[j++] = input[i++];
            result[j] = '\0';
        }
    }
    return result;
}

// Custom tokenizer that handles single quotes, double quotes, and plain words
int tokenize(char *input, char *args[], int *unmatched_quote_type) {
    int arg_count = 0;
    int i = 0;
    int len = strlen(input);
    *unmatched_quote_type = 0; // 0 = no error, 1 = single quote error, 2 = double quote error

    while (i < len) {
        // Skip whitespace
        while (i < len && (input[i] == ' ' || input[i] == '\t' || input[i] == '\n')) {
            i++;
        }
    }
}
```

```

    }
    if (i >= len) { // Unmatched double quote
        *unmatched_quote_type = 2;
        return -1;
    }
    strncpy(raw_token, &input[start], i - start);
    raw_token[i - start] = '\0';
    i++; // Skip closing quote

    // Expand variables inside double quotes
    char *expanded = expand_variable(raw_token);
    strcpy(token, expanded);
} else { // Standard Unquoted Token
    int start = i;
    while (i < len && input[i] != ' ' && input[i] != '\t' && input[i] != '\n' && input[i] != '\\' && input[i] != '"') {
        i++;
    }
    char raw_token[MAX_INPUT_LEN] = "";
    strncpy(raw_token, &input[start], i - start);
    raw_token[i - start] = '\0';

    // Allow variable expansion on unquoted words
    char *expanded = expand_variable(raw_token);
    strcpy(token, expanded);
}

args[arg_count] = strdup(token);
arg_count++;
}

args[arg_count] = NULL;
return arg_count;
}

int main() {
    char input[MAX_INPUT_LEN];

    while (1) {
        printf("myShell> ");
        fflush(stdout);

        // Read input
        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }

        // Check empty command
        if (input[0] == '\n' || input[0] == '\0') {
            continue;
        }

        char *args[MAX_ARGS];
        int quote_error = 0;
        int arg_count = tokenize(input, args, &quote_error);

        // Check for syntax errors
        if (arg_count == -1) {
            if (quote_error == 1) {
                printf("Syntax Error: Unmatched single quote.\n");
            } else if (quote_error == 2) {
                printf("Syntax Error: Unmatched double quote.\n");
            }
            continue;
        }

        // Handle case where input contains only spaces
        if (arg_count == 0) {
            continue;
        }
    }
}

```

```

}

// Check if double quotes were used to select output format style
int contains_double_quotes = (strchr(input, '"') != NULL);

if (contains_double_quotes) {
    printf("--- Parsed Arguments ---\n");
    for (int i = 0; i < arg_count; i++) {
        printf("Argument %d: %s\n", i + 1, args[i]);
    }
} else {
    printf("Parsing Result:\n");
    for (int i = 0; i < arg_count; i++) {
        printf("Argument %d: %s\n", i + 1, args[i]);
    }
}

// Process creation using fork()
pid_t pid = fork();

if (pid < 0) {
    perror("Fork failed");
    exit(1);
} else if (pid == 0) {
    // Child Process
    if (contains_double_quotes) {
        printf("Child Process\nPID = %d\n", getpid());
    } else {
        printf("Child PID: %d\n", getpid());
    }

    if (execvp(args[0], args) == -1) {
        perror("Execution failed");
    }
    exit(1);
} else {
    // Parent Process
    int status;
    waitpid(pid, &status, 0);

    if (contains_double_quotes) {
        printf("Child process completed.\n");
    } else {
        printf("Command execution completed.\n");
    }
}

// Free dynamically allocated tokens
for (int i = 0; i < arg_count; i++) {
    free(args[i]);
}

return 0;

```

```

imvarls@LAPTOP-VQ4D70L7:~$ nano myshell.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc myshell.c -o myshell

```

```
imvarls@LAPTOP-VQ4D70L7:~$ nano myshell.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc myshell.c -o myshell
imvarls@LAPTOP-VQ4D70L7:~$ ./myshell
myShell> |
```

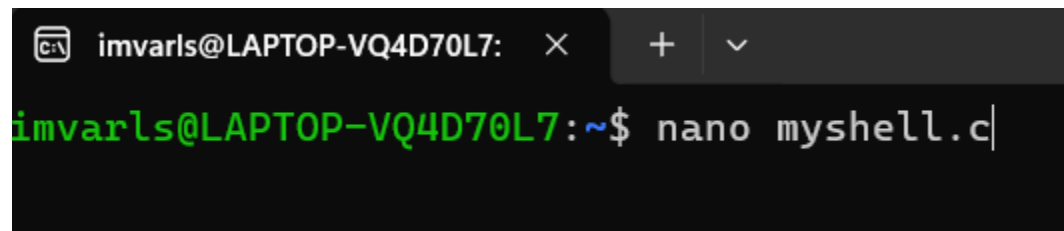
OUTPUT

```
imvarls@LAPTOP-VQ4D70L7:~$ nano myshell.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc myshell.c -o myshell
imvarls@LAPTOP-VQ4D70L7:~$ ./myshell
myShell> echo Hello
Parsing Result:
Argument 1: echo
Argument 2: Hello
Child PID: 625
Hello
Command execution completed.
myShell> |
```

```
myShell> 'Hello World'
Syntax Error: Unmatched single quote.
myShell> |
```

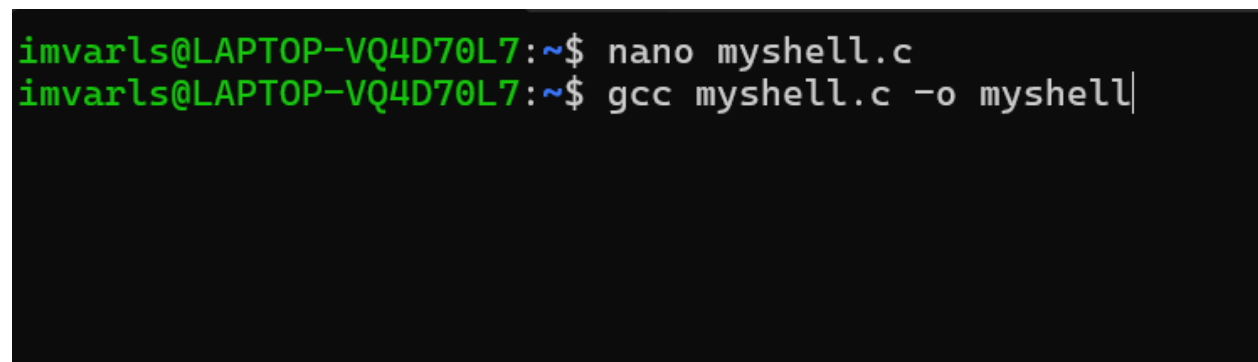
Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

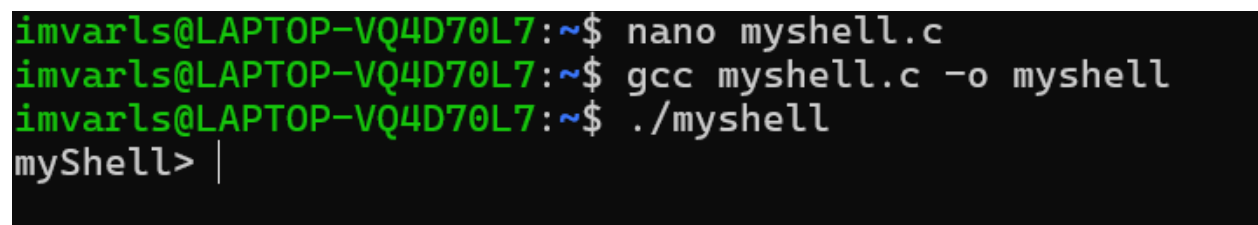


```
imvarls@LAPTOP-VQ4D70L7: ~$ nano myshell.c|
```

SAME CODE AS PREVIOUS ONE



```
imvarls@LAPTOP-VQ4D70L7: ~$ nano myshell.c
imvarls@LAPTOP-VQ4D70L7: ~$ gcc myshell.c -o myshell
```



```
imvarls@LAPTOP-VQ4D70L7: ~$ nano myshell.c
imvarls@LAPTOP-VQ4D70L7: ~$ gcc myshell.c -o myshell
imvarls@LAPTOP-VQ4D70L7: ~$ ./myshell
myShell> |
```

OUTPUT

```
myShell> echo "Hello $USER"  
--- Parsed Arguments ---  
Argument 1: echo  
Argument 2: Hello imvarls  
Child Process  
PID = 628  
Hello imvarls  
Child process completed.  
myShell> |
```

```
myShell> echo "HelloWorld  
Syntax Error: Unmatched double quote.  
myShell> |
```

